Chapter 14. Views

Well-designed applications generally expose a public interface while keeping implementation details private, thereby enabling future design changes without impacting end users. When designing your database, you can achieve a similar result by keeping your tables private and allowing your users to access data only through a set of *views*. This chapter strives to define what views are, how they are created, and when and how you might want to use them.

What Are Views?

A view is simply a mechanism for querying data. Unlike tables, views do not involve data storage; you won't need to worry about views filling up your disk space. You create a view by assigning a name to a `select` statement and then storing the query for others to use. Other users can then use your view to access data just as though they were querying tables directly (in fact, they may not even know they are using a view).

As a simple example, let's say that you want to partially obscure the email address in the `customer` table. The marketing department, for example, may need access to email addresses in order to advertise promotions, but otherwise your company's privacy policy dictates that this data be kept secure. Therefore, instead of allowing direct access to the `customer` table, you define a view called `customer_vw` and mandate that all nonmarketing personnel use it to access customer data. Here's the view definition:

```
CREATE VIEW customer_vw
 (customer_id,
  first_name,
  last_name,
  email
 )
AS
SELECT
  customer_id,
  first_name,
  last_name,
```

```
  concat(substr(email,1,2), '*****', substr(email, -4)) email
FROM customer;
```

The first part of the statement lists the view's column names, which may be different from those of the underlying table. The second part of the statement is a `select` statement, which must contain one expression for each column in the view. The `email` column is generated by taking the first two characters of the email address, concatenated with `'*****'`, and then concatenated with the last four characters of the email address.

When the `create view` statement is executed, the database server simply stores the view definition for future use; the query is not executed, and no data is retrieved or stored. Once the view has been created, users can query it just like they would a table, as in:

```
mysql> SELECT first_name, last_name, email
    -> FROM customer_vw;
+-------------+--------------+-------------+
| first_name  | last_name    | email       |
+-------------+--------------+-------------+
| MARY        | SMITH        | MA*****.org |
| PATRICIA    | JOHNSON      | PA*****.org |
| LINDA       | WILLIAMS     | LI*****.org |
| BARBARA     | JONES        | BA*****.org |
| ELIZABETH   | BROWN        | EL*****.org |
...
| ENRIQUE     | FORSYTHE     | EN*****.org |
| FREDDIE     | DUGGAN       | FR*****.org |
| WADE        | DELVALLE     | WA*****.org |
| AUSTIN      | CINTRON      | AU*****.org |
+-------------+--------------+-------------+
599 rows in set (0.00 sec)
```

Even though the `customer_vw` view definition includes four columns of the `customer` table, the previous query retrieves only three of the four. As you'll see later in the chapter, this is an important distinction if some of the columns in your view are attached to functions or subqueries.

From the user's standpoint, a view looks exactly like a table. If you want to know what columns are available in a view, you can use MySQL's (or Oracle's) `describe` command to examine it:

```
mysql> describe customer_vw;
+-------------+---------------------+------+-----+---------+-------+
| Field       | Type                | Null | Key | Default | Extra |
+-------------+---------------------+------+-----+---------+-------+
| customer_id | smallint(5) unsigned | NO   |     | 0       |       |
| first_name  | varchar(45)         | NO   |     | NULL    |       |
| last_name   | varchar(45)         | NO   |     | NULL    |       |
| email       | varchar(11)         | YES  |     | NULL    |       |
+-------------+---------------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

You are free to use any clauses of the `select` statement when querying through a view, including `group by`, `having`, and `order by`. Here's an example:

```
mysql> SELECT first_name, count(*), min(last_name), max(last_name)
    -> FROM customer_vw
    -> WHERE first_name LIKE 'J%'
    -> GROUP BY first_name
    -> HAVING count(*) > 1
    -> ORDER BY 1;
+------------+----------+---------------+---------------+
| first_name | count(*) | min(last_name) | max(last_name) |
+------------+----------+---------------+---------------+
| JAMIE      |        2 | RICE          | WAUGH         |
| JESSIE     |        2 | BANKS         | MILAM         |
+------------+----------+---------------+---------------+
2 rows in set (0.00 sec)
```

In addition, you can join views to other tables (or even to other views) within a query, as in:

```
mysql> SELECT cv.first_name, cv.last_name, p.amount
    -> FROM customer_vw cv
    ->   INNER JOIN payment p
    ->   ON cv.customer_id = p.customer_id
    -> WHERE p.amount >= 11;
+------------+-----------+--------+
| first_name | last_name | amount |
+------------+-----------+--------+
| KAREN      | JACKSON   |  11.99 |
| VICTORIA   | GIBSON    |  11.99 |
| VANESSA    | SIMS      |  11.99 |
```

```
| ALMA      | AUSTIN    |  11.99 |
| ROSEMARY  | SCHMIDT   |  11.99 |
| TANYA     | GILBERT   |  11.99 |
| RICHARD   | MCCRARY   |  11.99 |
| NICHOLAS  | BARFIELD  |  11.99 |
| KENT      | ARSENAULT |  11.99 |
| TERRANCE  | ROUSH     |  11.99 |
+-----------+-----------+--------+
10 rows in set (0.01 sec)
```

This query joins the `customer_vw` view to the `payment` table in order to find customers who have paid $11 or more for a film rental.

Why Use Views?

In the previous section, I demonstrated a simple view whose sole purpose was to mask the contents of the `customer.email` column. While views are often employed for this purpose, there are many reasons for using views, as detailed in the following subsections.

Data Security

If you create a table and allow users to query it, they will be able to access every column and every row in the table. As I pointed out earlier, however, your table may include some columns that contain sensitive data, such as identification numbers or credit card numbers; not only is it a bad idea to expose such data to all users, but also it might violate your company's privacy policies, or even state or federal laws, to do so.

The best approach for these situations is to keep the table private (i.e., don't grant `select` permission to any users) and then to create one or more views that either omit or obscure (such as the `'*****'` approach taken with the `customer_vw.email` column) the sensitive columns. You may also constrain which *rows* a set of users may access by adding a `where` clause to your view definition. For example, the next view definition excludes inactive customers:

```
CREATE VIEW active_customer_vw
 (customer_id,
  first_name,
  last_name,
  email
```

```
 )
AS
SELECT
  customer_id,
  first_name,
  last_name,
  concat(substr(email,1,2), '*****', substr(email, -4)) email
FROM customer
WHERE active = 1;
```

If you provide this view to your marketing department, they will be able to avoid sending information to inactive customers, because the condition in the view's `where` clause will always be included in their queries.

NOTE

Oracle Database users have another option for securing both rows and columns of a table: Virtual Private Database (VPD). VPD allows you to attach policies to your tables, after which the server will modify a user's query as necessary to enforce the policies. For example, if you enact a policy that members of the sales and marketing departments can see only active customers, then the condition `active = 1` will be added to all of their queries against the `customer` table.

Data Aggregation

Reporting applications generally require aggregated data, and views are a great way to make it appear as though data is being preaggregated and stored in the database. As an example, let's say that an application generates a report each month showing the total sales for each film category so that the managers can decide what new films to add to inventory. Rather than allowing the application developers to write queries against the base tables, you could provide them with the following view:[1]

```
CREATE VIEW sales_by_film_category
AS
SELECT
  c.name AS category,
  SUM(p.amount) AS total_sales
FROM payment AS p
  INNER JOIN rental AS r ON p.rental_id = r.rental_id
  INNER JOIN inventory AS i ON r.inventory_id = i.inventory_id
  INNER JOIN film AS f ON i.film_id = f.film_id
  INNER JOIN film_category AS fc ON f.film_id = fc.film_id
```

```
  INNER JOIN category AS c ON fc.category_id = c.category_id
GROUP BY c.name
ORDER BY total_sales DESC;
```

Using this approach gives you a great deal of flexibility as a database designer. If you decide at some point in the future that query performance would improve dramatically if the data were preaggregated in a table rather than summed using a view, you could create a `film_category_sales` table, load it with aggregated data, and modify the `sales_by_film_category` view definition to retrieve data from this table. Afterward, all queries that use the `sales_by_film_category` view will retrieve data from the new `film_category_sales` table, meaning that users will see a performance improvement without needing to modify their queries.

Hiding Complexity

One of the most common reasons for deploying views is to shield end users from complexity. For example, let's say that a report is created each month showing information about all of the films, along with the film category, the number of actors appearing in the film, the total number of copies in inventory, and the number of rentals for each film. Rather than expecting the report designer to navigate six different tables to gather the necessary data, you could provide a view that looks as follows:

```
CREATE VIEW film_stats
AS
SELECT f.film_id, f.title, f.description, f.rating,
 (SELECT c.name
  FROM category c
    INNER JOIN film_category fc
    ON c.category_id = fc.category_id
  WHERE fc.film_id = f.film_id) category_name,
 (SELECT count(*)
  FROM film_actor fa
  WHERE fa.film_id = f.film_id
 ) num_actors,
 (SELECT count(*)
  FROM inventory i
  WHERE i.film_id = f.film_id
 ) inventory_cnt,
 (SELECT count(*)
  FROM inventory i
    INNER JOIN rental r
    ON i.inventory_id = r.inventory_id
```

```
  WHERE i.film_id = f.film_id
 ) num_rentals
FROM film f;
```

This view definition is interesting because even though data from six different tables can be retrieved through the view, the `from` clause of the query has only one table (`film`). Data from the other five tables is generated using scalar subqueries. If someone uses this view but does *not* reference the `category_name`, `num_actors`, `inventory_cnt`, or `num_rentals` column, then none of the subqueries will be executed. This approach allows the view to be used for supplying descriptive information from the `film` table without unnecessarily joining five other tables.

Joining Partitioned Data

Some database designs break large tables into multiple pieces in order to improve performance. For example, if the `payment` table became large, the designers may decide to break it into two tables: `payment_current`, which holds the latest six months of data, and `payment_historic`, which holds all data up to six months ago. If a customer wants to see all the payments for a particular customer, you would need to query both tables. By creating a view that queries both tables and combines the results together, however, you can make it look like all payment data is stored in a single table. Here's the view definition:

```
CREATE VIEW payment_all
 (payment_id,
  customer_id,
  staff_id,
  rental_id,
  amount,
  payment_date,
  last_update
 )
AS
SELECT payment_id, customer_id, staff_id, rental_id,
  amount, payment_date, last_update
FROM payment_historic
UNION ALL
SELECT payment_id, customer_id, staff_id, rental_id,
  amount, payment_date, last_update
FROM payment_current;
```

Using a view in this case is a good idea because it allows the designers to change the structure of the underlying data without the need to force all database users to modify their queries.

Updatable Views

If you provide users with a set of views to use for data retrieval, what should you do if the users also need to modify the same data? It might seem a bit strange, for example, to force the users to retrieve data using a view but then allow them to directly modify the underlying table using `update` or `insert` statements. For this purpose, MySQL, Oracle Database, and SQL Server all allow you to modify data through a view, as long as you abide by certain restrictions. In the case of MySQL, a view is updatable if the following conditions are met:

- No aggregate functions are used (`max()` , `min()` , `avg()` , etc.).
- The view does not employ `group by` or `having` clauses.
- No subqueries exist in the `select` or `from` clause, and any subqueries in the `where` clause do not refer to tables in the `from` clause.
- The view does not utilize `union` , `union all` , or `distinct` .
- The `from` clause includes at least one table or updatable view.
- The `from` clause uses only inner joins if there is more than one table or view.

To demonstrate the utility of updatable views, it might be best to start with a simple view definition and then to move to a more complex view.

Updating Simple Views

The view at the beginning of the chapter is about as simple as it gets, so let's start there:

```
CREATE VIEW customer_vw
 (customer_id,
  first_name,
  last_name,
  email
 )
AS
SELECT
  customer_id,
  first_name,
  last_name,
```

```
  concat(substr(email,1,2), '*****', substr(email, -4)) email
FROM customer;
```

The `customer_vw` view queries a single table, and only one of the four columns is derived via an expression. This view definition doesn't violate any of the restrictions listed earlier, so you can use it to modify data in the `customer` table. Let's use the view to update Mary Smith's last name to Smith-Allen:

```
mysql> UPDATE customer_vw
    -> SET last_name = 'SMITH-ALLEN'
    -> WHERE customer_id = 1;
Query OK, 1 row affected (0.11 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

As you can see, the statement claims to have modified one row, but let's check the underlying `customer` table just to be sure:

```
mysql> SELECT first_name, last_name, email
    -> FROM customer
    -> WHERE customer_id = 1;
+------------+-------------+------------------------------+
| first_name | last_name   | email                        |
+------------+-------------+------------------------------+
| MARY       | SMITH-ALLEN | MARY.SMITH@sakilacustomer.org |
+------------+-------------+------------------------------+
1 row in set (0.00 sec)
```

While you can modify most of the columns in the view in this fashion, you will not be able to modify the `email` column, since it is derived from an expression:

```
mysql> UPDATE customer_vw
    -> SET email = 'MARY.SMITH-ALLEN@sakilacustomer.org'
    -> WHERE customer_id = 1;
ERROR 1348 (HY000): Column 'email' is not updatable
```

In this case, it may not be a bad thing, since the main reason for creating the view was to obscure the email addresses.

If you want to insert data using the `customer_vw` view, you are out of luck; views that contain derived columns cannot be used for inserting data, even if the derived columns are not included in the statement. For example, the next statement attempts to populate only the `customer_id`, `first_name`, and `last_name` columns using the `customer_vw` view:

```
mysql> INSERT INTO customer_vw
    ->  (customer_id,
    ->   first_name,
    ->   last_name)
    -> VALUES (99999,'ROBERT','SIMPSON');
ERROR 1471 (HY000): The target table customer_vw of the INSERT
is not insertable-into
```

Now that you have seen the limitations of simple views, the next section will demonstrate the use of a view that joins multiple tables.

Updating Complex Views

While single-table views are certainly common, many of the views that you come across will include multiple tables in the `from` clause of the underlying query. The next view, for example, joins the `customer`, `address`, `city`, and `country` tables so that all the data for customers can be easily queried:

```
CREATE VIEW customer_details
AS
SELECT c.customer_id,
  c.store_id,
  c.first_name,
  c.last_name,
  c.address_id,
  c.active,
  c.create_date,
  a.address,
  ct.city,
  cn.country,
  a.postal_code
FROM customer c
  INNER JOIN address a
  ON c.address_id = a.address_id
  INNER JOIN city ct
  ON a.city_id = ct.city_id
```

```
  INNER JOIN country cn
  ON ct.country_id = cn.country_id;
```

You may use this view to update data in either the `customer` or `address` table, as the following statements demonstrate:

```
mysql> UPDATE customer_details
    -> SET last_name = 'SMITH-ALLEN', active = 0
    -> WHERE customer_id = 1;
Query OK, 1 row affected (0.10 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE customer_details
    -> SET address = '999 Mockingbird Lane'
    -> WHERE customer_id = 1;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

The first statement modifies the `customer.last_name` and `customer.active` columns, whereas the second statement modifies the `address.address` column. You might be wondering what happens if you try to update columns from *both* tables in a single statement, so let's find out:

```
mysql> UPDATE customer_details
    -> SET last_name = 'SMITH-ALLEN',
    ->   active = 0,
    ->   address = '999 Mockingbird Lane'
    -> WHERE customer_id = 1;
ERROR 1393 (HY000): Can not modify more than one base table
  through a join view 'sakila.customer_details'
```

As you can see, you are allowed to modify both of the underlying tables separately, but not within a single statement. Next, let's try to *insert* data into both tables for some new customers (`customer_id = 9998 and 9999`):

```
mysql> INSERT INTO customer_details
    ->  (customer_id, store_id, first_name, last_name,
    ->   address_id, active, create_date)
    -> VALUES (9998, 1, 'BRIAN', 'SALAZAR', 5, 1, now());
Query OK, 1 row affected (0.23 sec)
```

This statement, which only populates columns from the `customer` table, works fine. Let's see what happens if we expand the column list to also include a column from the `address` table:

```
mysql> INSERT INTO customer_details
    ->  (customer_id, store_id, first_name, last_name,
    ->   address_id, active, create_date, address)
    -> VALUES (9999, 2, 'THOMAS', 'BISHOP', 7, 1, now(),
    ->  '999 Mockingbird Lane');
ERROR 1393 (HY000): Can not modify more than one base table
  through a join view 'sakila.customer_details'
```

This version, which includes columns spanning two different tables, raises an exception. In order to insert data through a complex view, you would need to know from where each column is sourced. Since many views are created to hide complexity from end users, this seems to defeat the purpose if the users need to have explicit knowledge of the view definition.

NOTE

Oracle Database and SQL Server also allow data to be inserted and updated through views, but, like MySQL, there are many restrictions. If you are willing to write some PL/SQL or Transact-SQL, however, you can use a feature called *instead-of triggers*, which allows you to essentially intercept `insert`, `update`, and `delete` statements against a view and write custom code to incorporate the changes. Without this type of feature, there are usually too many restrictions to make updating through views a feasible strategy for nontrivial applications.

Test Your Knowledge

Test your understanding of views by working through the following exercises. When you're done, compare your solutions with those in Appendix B.

Exercise 14-1

Create a view definition that can be used by the following query to generate the given results:

```
SELECT title, category_name, first_name, last_name
FROM film_ctgry_actor
```

```
WHERE last_name = 'FAWCETT';

+--------------------+--------------+------------+-----------+
| title              | category_name | first_name | last_name |
+--------------------+--------------+------------+-----------+
| ACE GOLDFINGER     | Horror       | BOB        | FAWCETT   |
| ADAPTATION HOLES   | Documentary  | BOB        | FAWCETT   |
| CHINATOWN GLADIATOR | New         | BOB        | FAWCETT   |
| CIRCUS YOUTH       | Children     | BOB        | FAWCETT   |
| CONTROL ANTHEM     | Comedy       | BOB        | FAWCETT   |
| DARES PLUTO        | Animation    | BOB        | FAWCETT   |
| DARN FORRESTER     | Action       | BOB        | FAWCETT   |
| DAZED PUNK         | Games        | BOB        | FAWCETT   |
| DYNAMITE TARZAN    | Classics     | BOB        | FAWCETT   |
| HATE HANDICAP      | Comedy       | BOB        | FAWCETT   |
| HOMICIDE PEACH     | Family       | BOB        | FAWCETT   |
| JACKET FRISCO      | Drama        | BOB        | FAWCETT   |
| JUMANJI BLADE      | New          | BOB        | FAWCETT   |
| LAWLESS VISION     | Animation    | BOB        | FAWCETT   |
| LEATHERNECKS DWARFS | Travel      | BOB        | FAWCETT   |
| OSCAR GOLD         | Animation    | BOB        | FAWCETT   |
| PELICAN COMFORTS   | Documentary  | BOB        | FAWCETT   |
| PERSONAL LADYBUGS  | Music        | BOB        | FAWCETT   |
| RAGING AIRPLANE    | Sci-Fi       | BOB        | FAWCETT   |
| RUN PACIFIC        | New          | BOB        | FAWCETT   |
| RUNNER MADIGAN     | Music        | BOB        | FAWCETT   |
| SADDLE ANTITRUST   | Comedy       | BOB        | FAWCETT   |
| SCORPION APOLLO    | Drama        | BOB        | FAWCETT   |
| SHAWSHANK BUBBLE   | Travel       | BOB        | FAWCETT   |
| TAXI KICK          | Music        | BOB        | FAWCETT   |
| BERETS AGENT       | Action       | JULIA      | FAWCETT   |
| BOILED DARES       | Travel       | JULIA      | FAWCETT   |
| CHISUM BEHAVIOR    | Family       | JULIA      | FAWCETT   |
| CLOSER BANG        | Comedy       | JULIA      | FAWCETT   |
| DAY UNFAITHFUL     | New          | JULIA      | FAWCETT   |
| HOPE TOOTSIE       | Classics     | JULIA      | FAWCETT   |
| LUKE MUMMY         | Animation    | JULIA      | FAWCETT   |
| MULAN MOON         | Comedy       | JULIA      | FAWCETT   |
| OPUS ICE           | Foreign      | JULIA      | FAWCETT   |
| POLLOCK DELIVERANCE | Foreign     | JULIA      | FAWCETT   |
| RIDGEMONT SUBMARINE | New         | JULIA      | FAWCETT   |
| SHANGHAI TYCOON    | Travel       | JULIA      | FAWCETT   |
| SHAWSHANK BUBBLE   | Travel       | JULIA      | FAWCETT   |
| THEORY MERMAID     | Animation    | JULIA      | FAWCETT   |
| WAIT CIDER         | Animation    | JULIA      | FAWCETT   |
+--------------------+--------------+------------+-----------+
```

```
40 rows in set (0.00 sec)
```

Exercise 14-2

The film rental company manager would like to have a report that includes the name of every country, along with the total payments for all customers who live in each country. Generate a view definition that queries the `country` table and uses a scalar subquery to calculate a value for a column named `tot_payments`.

1 This view definition is included in the Sakila sample database, along with six others, several of which will be used in upcoming examples.

Previous chapter
< 13. Indexes and Constraints

Next chapter
15. Metadata >